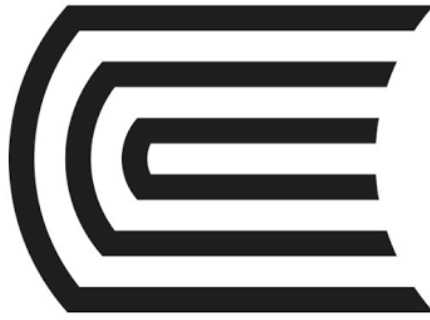




# **Universidad Continental**

ASIGNATURA: ESTRUCTURA DE DATOS

DOCENTE: Rosario Delia Osorio

## **Integrantes:**

1. Sota Quispe Sebastian Yojan
2. Cerron Lopez Hans Leonardo
3. Anchiraico Aylas Joseph Jesús
4. Ortega Matos Andy Jeampier

Huancayo - 2026

# **Capítulo 1**

Análisis del problema

## 1.1 Descripción del problema

En la actualidad, los sistemas operativos deben administrar múltiples procesos que se ejecutan al mismo tiempo. Estos procesos compiten por recursos como el procesador y la memoria, por lo que es necesario contar con mecanismos que permitan organizarlos y gestionarlos de manera eficiente.

El presente proyecto propone desarrollar un Sistema de Gestión de Procesos que simule algunas funciones básicas de un administrador de tareas. El sistema permitirá registrar procesos, organizarlos según su prioridad, controlar la memoria utilizada y mantener un historial de las tareas ejecutadas.

Para el desarrollo de la solución se utilizarán estructuras de datos dinámicas lineales implementadas desde cero en C ++, específicamente listas enlazadas simples, colas y pilas. Asimismo, el sistema contará con persistencia de datos mediante archivos de texto, permitiendo conservar la información registrada incluso después de cerrar la aplicación.

## 1.2 Requerimientos del sistema

### Requerimientos funcionales

#### RF-01. Gestión de procesos (Lista Enlazada simples)

- Registrar nuevos procesos ingresando ID, nombre, prioridad y cantidad de memoria asignada.
- Mostrar la lista completa de procesos registrados.
- Buscar procesos mediante su ID o nombre.
- Modificar la prioridad de un proceso cuando sea necesario.
- Eliminar procesos del sistema liberando la memoria correspondiente.
- Validar que no existan IDs duplicados.

#### RF-02. Planificación de CPU (Cola de Prioridad)

- Organizar los procesos según su nivel de prioridad.
- Mantener el orden de llegada para los procesos que tengan la misma prioridad.
- Ejecutar procesos respetando las prioridades establecidas.
- Mostrar los procesos pendientes de ejecución.

#### RF-03. Gestión de memoria e historial (Pila)

- Registrar los procesos finalizados en una pila de historial.
- Liberar memoria retirando elementos desde el tope de la pila.

- Consultar el historial de procesos ejecutados.
- Mostrar los procesos almacenados desde el más reciente hasta el más antiguo.

#### RF-04. Persistencia de datos

- Guardar automáticamente la información del sistema en archivos de texto antes de finalizar la ejecución.
- Cargar los datos almacenados al iniciar el programa para reconstruir las estructuras dinámicas.

### Requerimientos no funcionales

#### RNF-01. Implementación de estructuras

- Las estructuras de datos deberán desarrollarse desde cero sin utilizar la librería STL.
- Se emplearán punteros y memoria dinámica mediante los operadores new y delete.

#### RNF-02. Compatibilidad

- El sistema deberá ejecutarse correctamente en el entorno Dev-C++.
- El programa funcionará mediante una interfaz de consola de fácil uso.

#### RNF-03. Gestión de archivos

- Se utilizará la librería fstream para guardar y recuperar la información almacenada.
- El sistema deberá validar la existencia de los archivos antes de intentar cargarlos.

#### RNF-04. Confiabilidad

- Se validará el ingreso de datos para evitar errores durante la ejecución.
- El sistema deberá impedir el registro de procesos con identificadores repetidos.

## 1.3 Estructuras de datos propuestas

### Lista Enlazada Simple (Gestor de Procesos)

Se utilizará para almacenar todos los procesos registrados en el sistema. Cada nodo contendrá información como ID, nombre, prioridad, memoria asignada y estado.

Esta estructura permitirá registrar, buscar, modificar y eliminar procesos de forma dinámica, facilitando la administración de la información.

### **Cola de Prioridad (Planificador de CPU)**

Se utilizará para organizar los procesos según su nivel de prioridad. Los procesos con mayor prioridad serán atendidos primero y aquellos con la misma prioridad conservarán el orden de llegada.

Esta estructura permitirá administrar y visualizar los procesos pendientes de ejecución, simulando el funcionamiento básico de un planificador de CPU.

### **Pila (Gestor de Memoria)**

Se utilizará para almacenar los procesos finalizados y apoyar la gestión de memoria del sistema.

Su funcionamiento se basa en el principio LIFO (Último en entrar, primero en salir), permitiendo consultar los procesos más recientes y simular la liberación de memoria.

### **Persistencia de Datos**

El sistema permitirá guardar y recuperar información mediante archivos de texto, evitando la pérdida de datos entre distintas ejecuciones del programa.

## **1.4 Justificación de la elección**

Las estructuras de datos seleccionadas fueron elegidas porque se adaptan a las necesidades de cada módulo del sistema y permiten representar de manera sencilla algunas funciones básicas de un sistema operativo.

### **Lista Enlazada Simple (Gestor de Procesos)**

La lista enlazada simple fue seleccionada porque permite almacenar procesos de forma dinámica sin establecer un tamaño fijo desde el inicio. Además, facilita operaciones como inserción, búsqueda, modificación y eliminación de información.

### **Cola de Prioridad (Planificador de CPU)**

La cola de prioridad fue elegida porque permite organizar los procesos según su importancia dentro del sistema. De esta manera, los procesos más prioritarios podrán ejecutarse antes que aquellos con menor prioridad.

### **Pila (Gestor de Memoria)**

La pila fue seleccionada porque su funcionamiento se adapta al registro de procesos finalizados y a la simulación de liberación de memoria. Su principio LIFO permite acceder rápidamente a los elementos más recientes.

## **Persistencia de Datos**

La persistencia de datos permitirá conservar la información del sistema entre diferentes sesiones de uso, evitando la pérdida de información y facilitando la reconstrucción de las estructuras al iniciar nuevamente el programa.

# **CAPÍTULO 2**

## **Diseño de la solución**

## 2.1 Descripción de Estructuras de Datos y Operaciones

Para el desarrollo del sistema se usarán estructuras de datos dinámicas lineales implementadas mediante una asignación dinámica de memoria. Esto nos ayuda a administrar los procesos de manera eficiente, ya que la memoria utilizada puede adaptarse según la cantidad de información almacenada durante la ejecución del programa.

### 2.1.1 Lista Enlazada Simple (Gestión de Procesos)

La lista enlazada simple será la estructura principal del sistema y se utilizará para almacenar todos los procesos registrados. Cada nodo almacenará la información de un proceso y tendrá un enlace al siguiente nodo, permitiendo recorrer la lista de manera secuencial.

|                                    |                                                                                                                                                                                                                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Datos<br/>almacenados</b>       | ID: Código identificador del proceso<br><br>Nombre: Nombre del proceso registrado<br><br>Prioridad: Nivel de prioridad asignado<br><br>Estado: Estado actual del proceso (Listo, Ejecutando o Finalizado)<br><br>Siguiente: Puntero al siguiente nodo de la lista |
| <b>Operaciones<br/>principales</b> | Agregar procesos al final de la lista, buscar procesos por ID o nombre, modificar la prioridad de un proceso, eliminar procesos y mostrar la lista completa de procesos.                                                                                          |

### 2.1.2 Cola de Prioridad (Planificador de CPU)

La cola de prioridad se utilizará para organizar los procesos que esperan ser ejecutados por el CPU. A diferencia de una cola tradicional FIFO (Primero en Entrar, Primero en Salir) los procesos se insertarán según su nivel de prioridad, permitiendo que aquellos con mayor prioridad sean atendidos primero.

|                                    |                                                                                                                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Datos<br/>almacenados</b>       | ID: Código identificador del proceso<br><br>Nombre: Nombre del proceso<br><br>Prioridad: Nivel de prioridad utilizado para el ordenamiento<br><br>Siguiente: Puntero al siguiente nodo de la cola |
| <b>Operaciones<br/>principales</b> | Insertar procesos según su prioridad, extraer el proceso ubicado al frente de la cola para simular su ejecución y mostrar los procesos pendientes de ejecución.                                   |

### 2.1.3 Pila (Gestión de Memoria)

La pila se utilizará para simular la asignación de memoria de los procesos dentro del sistema. Esta estructura sigue el principio LIFO (Último en Entrar, Primero en Salir) donde el último elemento ingresado es el primero en salir.

|                                    |                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Datos<br/>almacenados</b>       | ID: Código identificador del proceso propietario<br><br>Nombre: Nombre del proceso asociado<br><br>Tamaño: Espacio de memoria utilizado en KB<br><br>Siguiente: Puntero al siguiente nodo de la pila |
| <b>Operaciones<br/>principales</b> | Asignar memoria mediante la operación Push, liberar memoria mediante la operación Pop, mostrar el estado actual de la memoria simulada y calcular la memoria total utilizada.                        |

### Algoritmos de las Operaciones y Análisis de las Estructuras de Control

El análisis de los algoritmos se realizará considerando las estructuras de control utilizadas durante la implementación del sistema. Para ello se describirá el uso de ciclos, condicionales y funciones en las principales operaciones desarrolladas.

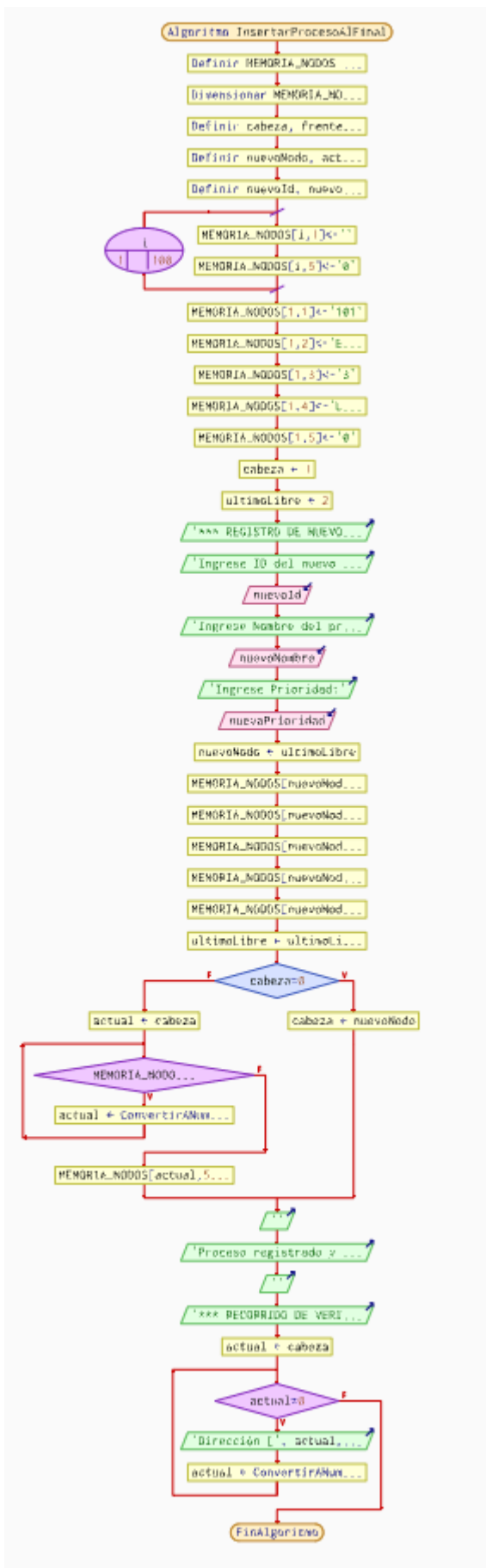
## Gestor de procesos (Lista enlazada)

### Pseudocódigo: Insertar Proceso al Fin

Proceso InsertarProcesoAlFinal

```
Definir MEMORIA_NODOS Como Cadena
Dimensionar MEMORIA_NODOS(100,5)
Definir cabeza, frente, tope, ultimoLibre, i Como Entero
Definir nuevoNodo, actual Como Entero
Definir nuevold, nuevoNombre, nuevaPrioridad Como Cadena
Para i<-1 Hasta 100 Hacer
    MEMORIA_NODOS[i,1]<-''
    MEMORIA_NODOS[i,5]<-0'
FinPara
MEMORIA_NODOS[1,1]<-101'
MEMORIA_NODOS[1,2]<-Excel'
MEMORIA_NODOS[1,3]<-3'
MEMORIA_NODOS[1,4]<-LISTO'
MEMORIA_NODOS[1,5]<-0'
cabeza <- 1
ultimoLibre <- 2
Escribir '*** REGISTRO DE NUEVO PROCESO (AL FINAL) ***'
Escribir 'Ingrese ID del nuevo proceso:'
Leer nuevold
Escribir 'Ingrese Nombre del proceso:'
Leer nuevoNombre
Escribir 'Ingrese Prioridad:'
Leer nuevaPrioridad
nuevoNodo <- ultimoLibre
MEMORIA_NODOS[nuevoNodo,1]<-nuevold
MEMORIA_NODOS[nuevoNodo,2]<-nuevoNombre
MEMORIA_NODOS[nuevoNodo,3]<-nuevaPrioridad
MEMORIA_NODOS[nuevoNodo,4]<-LISTO'
MEMORIA_NODOS[nuevoNodo,5]<-0'
ultimoLibre <- ultimoLibre+1
Si cabeza=0 Entonces
    cabeza <- nuevoNodo
SiNo
    actual <- cabeza
    Mientras MEMORIA_NODOS[actual,5]<>'0' Hacer
        actual <- ConvertirANumero(MEMORIA_NODOS[actual,5])
    FinMientras
    MEMORIA_NODOS[actual,5]<-ConvertirATexto(nuevoNodo)
FinSi
Escribir ''
Escribir 'Proceso registrado y enlazado exitosamente al final.'
Escribir ''
Escribir '*** RECORRIDO DE VERIFICACIÓN ***'
actual <- cabeza
Mientras actual<>0 Hacer
    Escribir 'Dirección [, actual, ] -> ID: ', MEMORIA_NODOS[actual,1], ' | Nombre: ',
MEMORIA_NODOS[actual,2], ' | Siguiente apunta a: ', MEMORIA_NODOS[actual,5]
    actual <- ConvertirANumero(MEMORIA_NODOS[actual,5])
FinMientras
```

FinProceso



## Pseudocódigo: Eliminar Proceso por ID

Proceso EliminarProcesoLista

Definir MEMORIA\_NODOS Como Cadena  
Dimension MEMORIA\_NODOS[100, 5]  
Definir cabeza, frente, tope, ultimoLibre, i Como Entero  
Definir actual, anterior Como Entero  
Definir encontrado Como Logico  
Definir idTarget Como Cadena

Para i <- 1 Hasta 100 Hacer  
    MEMORIA\_NODOS[i, 1] <- ""  
    MEMORIA\_NODOS[i, 5] <- "0"  
FinPara

MEMORIA\_NODOS[1, 1] <- "101"  
MEMORIA\_NODOS[1, 2] <- "Word"  
MEMORIA\_NODOS[1, 5] <- "2"  
MEMORIA\_NODOS[2, 1] <- "102"  
MEMORIA\_NODOS[2, 2] <- "Excel"  
MEMORIA\_NODOS[2, 5] <- "0"

cabeza <- 1  
ultimoLibre <- 3

Escribir "Ingrese ID a eliminar (Prueba con 101 o 102):"  
Leer idTarget

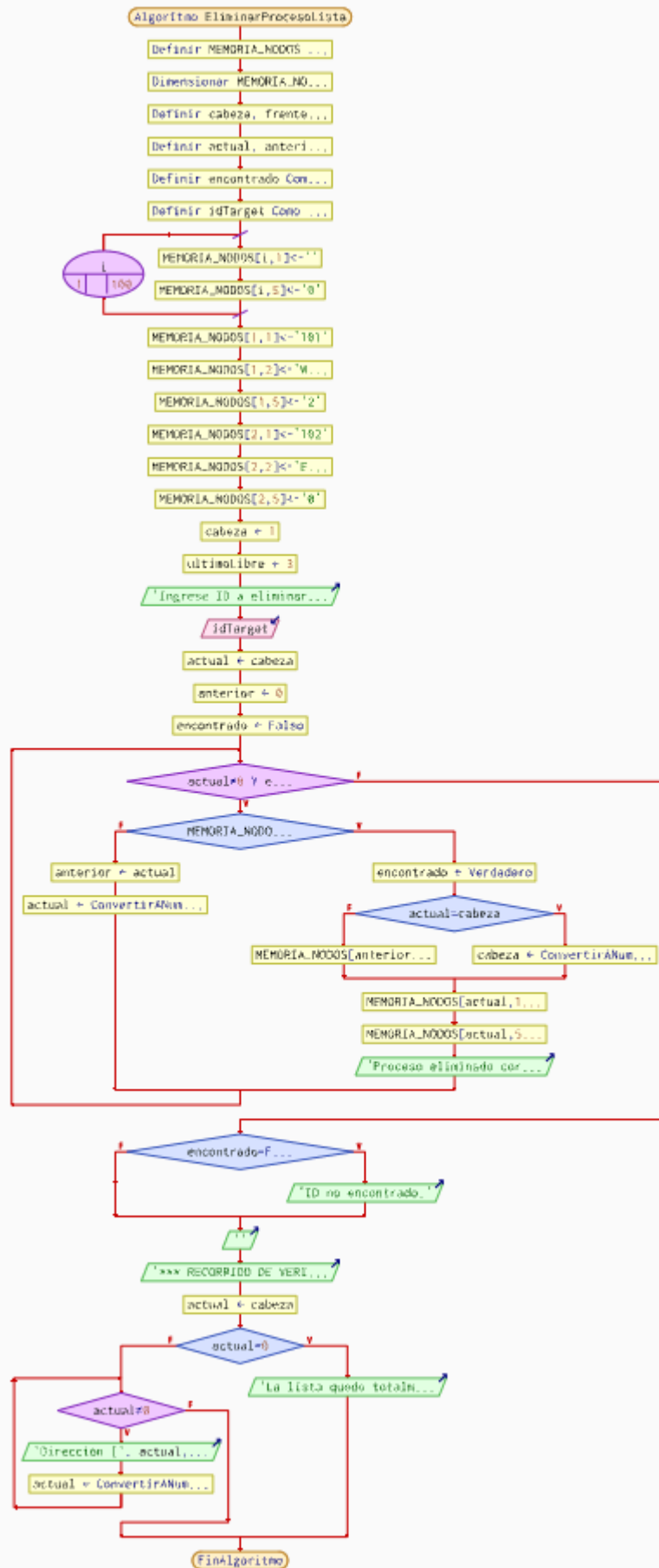
actual <- cabeza  
anterior <- 0  
encontrado <- Falso

Mientras actual <> 0 Y encontrado = Falso Hacer  
    Si MEMORIA\_NODOS[actual, 1] = idTarget Entonces  
        encontrado <- Verdadero  
        Si actual = cabeza Entonces  
            cabeza <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])  
        Sino  
            MEMORIA\_NODOS[anterior, 5] <- MEMORIA\_NODOS[actual, 5]  
    FinSi  
    MEMORIA\_NODOS[actual, 1] <- ""  
    MEMORIA\_NODOS[actual, 5] <- "0"  
    Escribir "Proceso eliminado correctamente de la memoria."  
    Sino  
        anterior <- actual  
        actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])  
    FinSi  
FinMientras

Si encontrado = Falso Entonces  
    Escribir "ID no encontrado."  
FinSi

Escribir ""  
Escribir "\*\*\*\* RECORRIDO DE VERIFICACIÓN \*\*\*\*"  
actual <- cabeza  
Si actual = 0 Entonces  
    Escribir "La lista quedo totalmente vacia (Cabeza apunta a 0)"  
Sino

    Mientras actual <> 0 Hacer  
        Escribir "Dirección [", actual, "] -> ID: ", MEMORIA\_NODOS[actual, 1], " | Nombre: ", MEMORIA\_NODOS[actual, 2], " |  
Siguiete apunta a: ", MEMORIA\_NODOS[actual, 5]  
        actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])  
    FinMientras  
FinSi  
FinProceso



## Pseudocódigo: Buscar Proceso por ID o Nombre

Proceso BuscarProcesoLista

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir actual Como Entero

Definir buscado Como Cadena

Definir encontrado Como Logico

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "101"

MEMORIA\_NODOS[1, 2] <- "Word"

MEMORIA\_NODOS[1, 4] <- "LISTO"

MEMORIA\_NODOS[1, 5] <- "0"

cabeza <- 1

Escribir "Ingrese ID o Nombre a buscar (Prueba con 101 o Word):"

Leer buscado

actual <- cabeza

encontrado <- Falso

Escribir ""

Escribir "\*\*\* RECORRIDO DE VERIFICACIÓN \*\*\*"

Mientras actual <> 0 Hacer

Si MEMORIA\_NODOS[actual, 1] = buscado O MEMORIA\_NODOS[actual, 2] = buscado Entonces

Escribir "COINCIDENCIA ENCONTRADA"

Escribir "Dirección [", actual, "] -> ID: ", MEMORIA\_NODOS[actual, 1], " | Nombre: ", MEMORIA\_NODOS[actual, 2], " | Estado: ", MEMORIA\_NODOS[actual, 4]

encontrado <- Verdadero

Sino

Escribir "Revisando Dirección [", actual, "]... No coincide."

FinSi

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

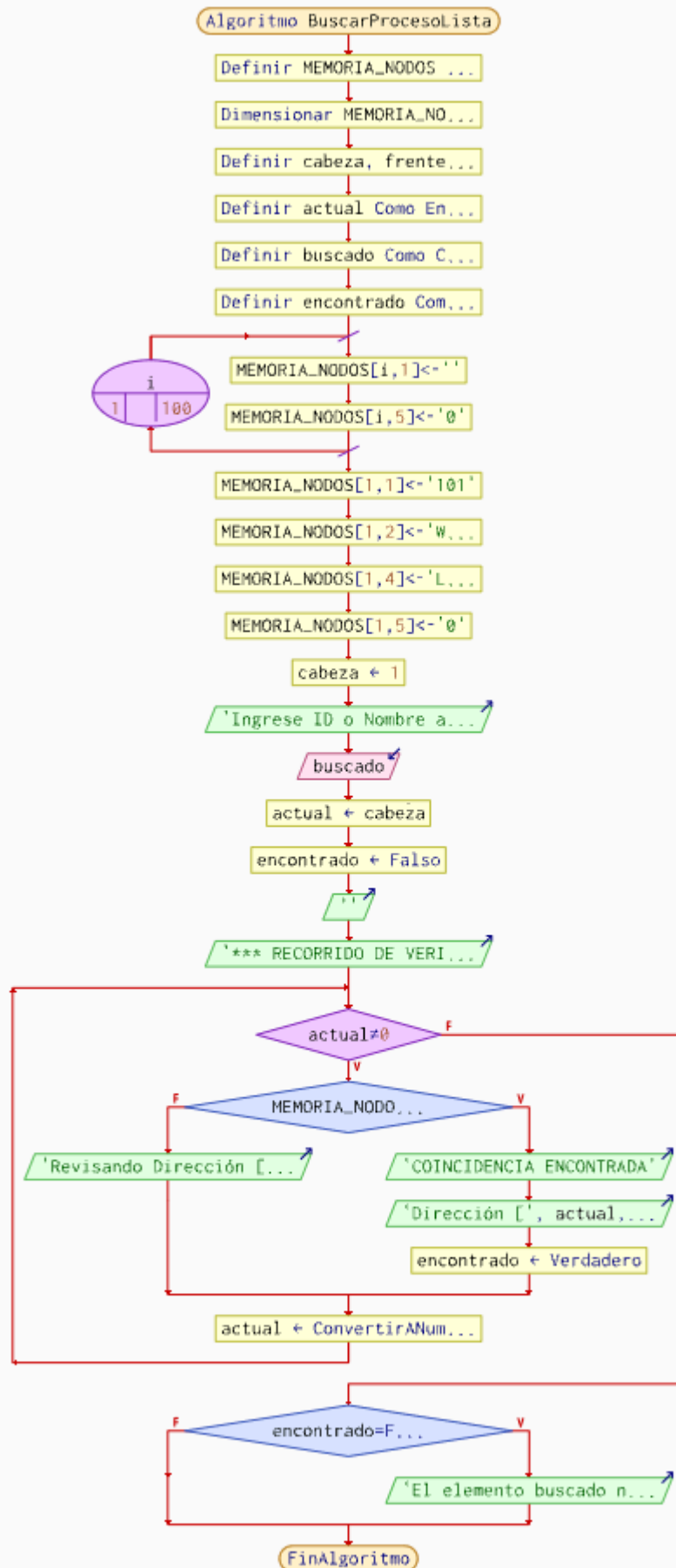
FinMientras

Si encontrado = Falso Entonces

Escribir "El elemento buscado no existe en la lista enlazada."

FinSi

FinProceso



## Pseudocódigo: Modificar Prioridad de un Proceso

Proceso ModificarPrioridadLista

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir actual Como Entero

Definir idTarget, nuevaPrioridad Como Cadena

Definir encontrado Como Logico

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "101"

MEMORIA\_NODOS[1, 2] <- "Word"

MEMORIA\_NODOS[1, 3] <- "3"

MEMORIA\_NODOS[1, 5] <- "0"

cabeza <- 1

Escribir "Ingrese ID del proceso a modificar (Prueba con 101):"

Leer idTarget

actual <- cabeza

encontrado <- Falso

Mientras actual <> 0 Y encontrado = Falso Hacer

Si MEMORIA\_NODOS[actual, 1] = idTarget Entonces

encontrado <- Verdadero

Escribir "Proceso encontrado: ", MEMORIA\_NODOS[actual, 2]

Escribir "Ingrese nueva prioridad:"

Leer nuevaPrioridad

MEMORIA\_NODOS[actual, 3] <- nuevaPrioridad

Escribir "Prioridad modificada con éxito."

Sino

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinSi

FinMientras

Si encontrado = Falso Entonces

Escribir "Proceso no existe."

FinSi

Escribir ""

Escribir "\*\*\*\* RECORRIDO DE VERIFICACIÓN \*\*\*\*"

actual <- cabeza

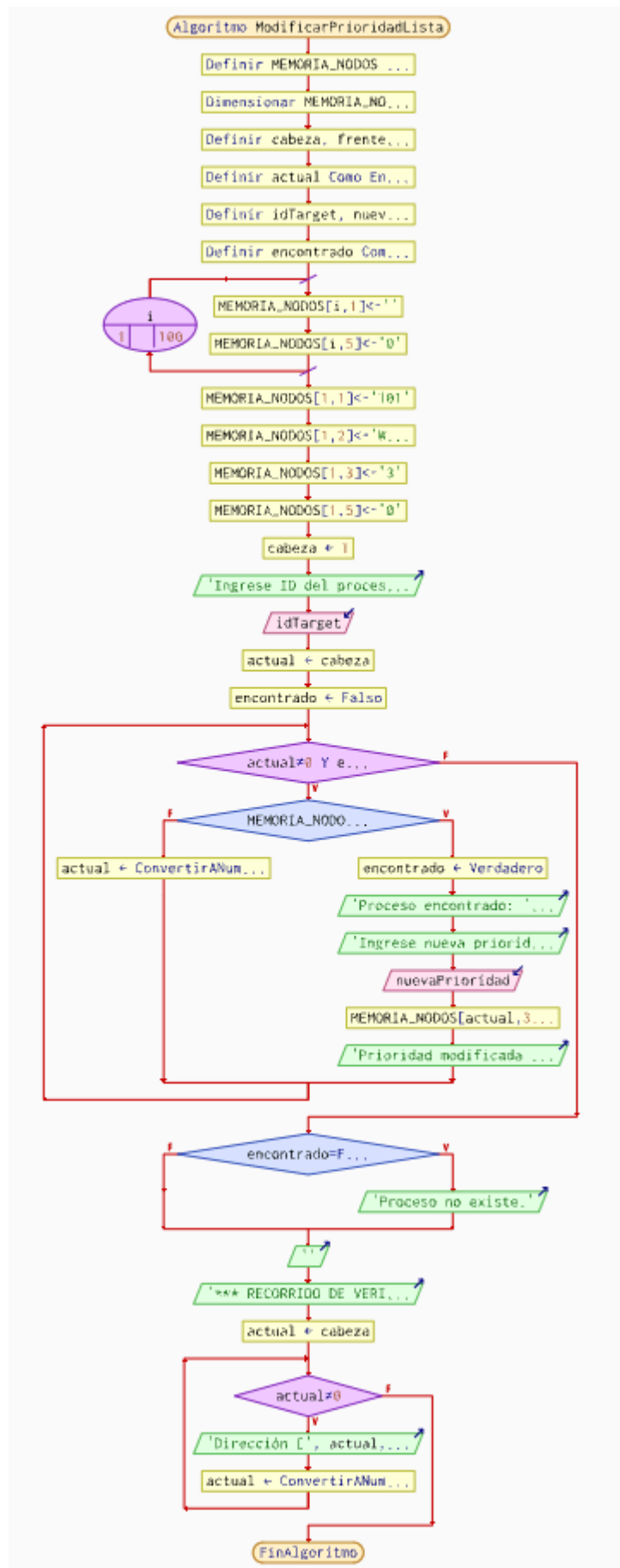
Mientras actual <> 0 Hacer

Escribir "Dirección [", actual, "] -> ID: ", MEMORIA\_NODOS[actual, 1], " | Nombre: ",  
MEMORIA\_NODOS[actual, 2], " | NUEVA Prioridad: ", MEMORIA\_NODOS[actual, 3]

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

FinProceso



## Pseudocódigo: Listar Todos los Procesos Registrados

Proceso ListarProcesosLista

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir actual Como Entero

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "101"

MEMORIA\_NODOS[1, 2] <- "Word"

MEMORIA\_NODOS[1, 3] <- "4"

MEMORIA\_NODOS[1, 4] <- "LISTO"

MEMORIA\_NODOS[1, 5] <- "2"

MEMORIA\_NODOS[2, 1] <- "102"

MEMORIA\_NODOS[2, 2] <- "Excel"

MEMORIA\_NODOS[2, 3] <- "5"

MEMORIA\_NODOS[2, 4] <- "EJECUTANDO"

MEMORIA\_NODOS[2, 5] <- "0"

cabeza <- 1

Escribir "\*\*\*\* RECORRIDO DE VERIFICACIÓN (LISTADO) \*\*\*\*"

Si cabeza = 0 Entonces

Escribir "La lista está vacía."

Sino

actual <- cabeza

Mientras actual <> 0 Hacer

Escribir "Dirección [", actual, "] -> ID: ", MEMORIA\_NODOS[actual, 1], " |

Tarea: ", MEMORIA\_NODOS[actual, 2], " | Prioridad: ",

MEMORIA\_NODOS[actual, 3], " | Siguiente apunta a: ",

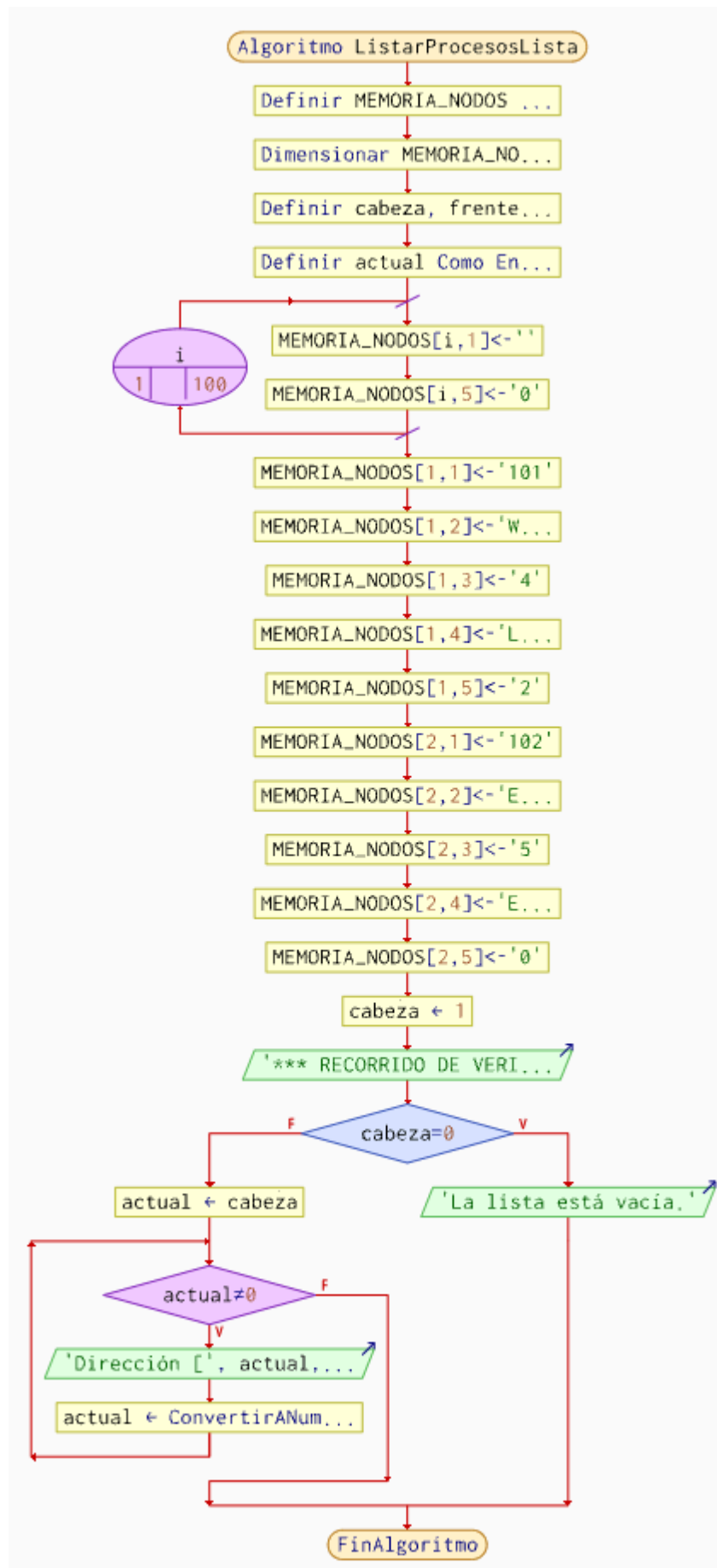
MEMORIA\_NODOS[actual, 5]

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

FinSi

FinProceso



## PLANIFICADOR DE CPU (COLA DE PRIORIDAD DINÁMICA)

### Pseudocódigo: Encolar Procesos según Prioridad

```
Proceso EncolarConPrioridad
  Definir MEMORIA_NODOS Como Cadena
  Dimension MEMORIA_NODOS[100, 5]
  Definir cabeza, frente, tope, ultimoLibre, i Como Entero
  Definir nuevoNode, actual, anterior Como Entero
  Definir nuevold, nuevoNombre, nuevaPrioridad Como Cadena

  Para i <- 1 Hasta 100 Hacer
    MEMORIA_NODOS[i, 1] <- ""
    MEMORIA_NODOS[i, 5] <- "0"
  FinPara

  MEMORIA_NODOS[1, 1] <- "101"
  MEMORIA_NODOS[1, 2] <- "Word"
  MEMORIA_NODOS[1, 3] <- "5"
  MEMORIA_NODOS[1, 5] <- "0"

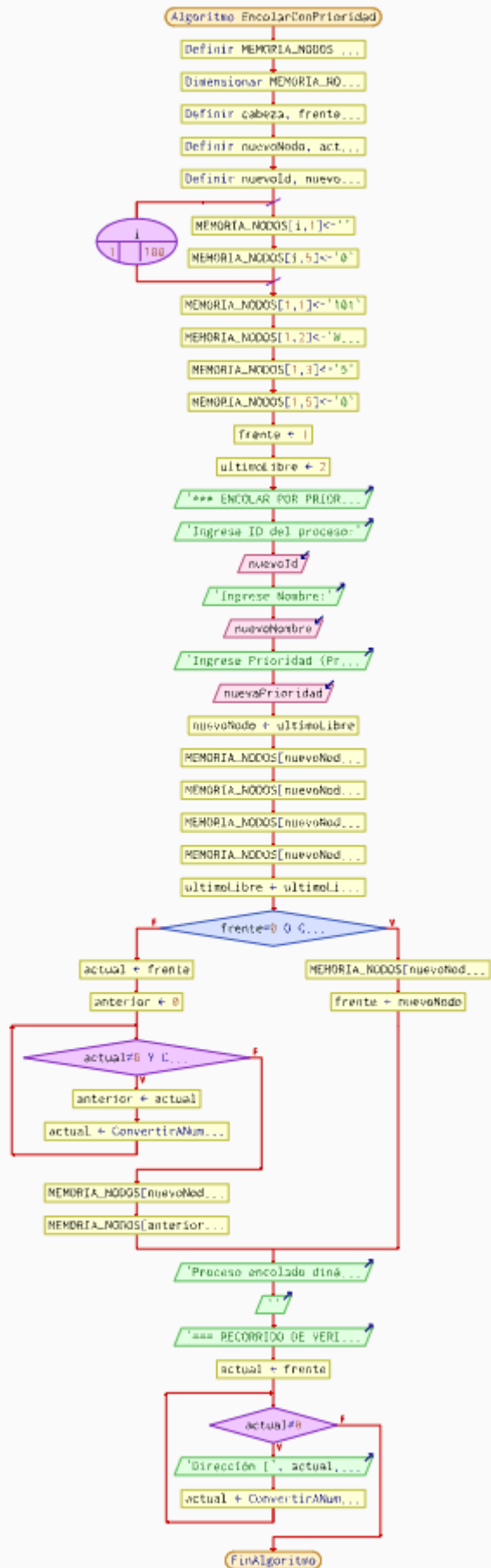
  frente <- 1
  ultimoLibre <- 2

  Escribir "**** ENCOLAR POR PRIORIDAD ****"
  Escribir "Ingrese ID del proceso:"
  Leer nuevold
  Escribir "Ingrese Nombre:"
  Leer nuevoNombre
  Escribir "Ingrese Prioridad (Prueba con un numero menor a 5 o mayor a 5):"
  Leer nuevaPrioridad

  nuevoNode <- ultimoLibre
  MEMORIA_NODOS[nuevoNode, 1] <- nuevold
  MEMORIA_NODOS[nuevoNode, 2] <- nuevoNombre
  MEMORIA_NODOS[nuevoNode, 3] <- nuevaPrioridad
  MEMORIA_NODOS[nuevoNode, 5] <- "0"
  ultimoLibre <- ultimoLibre + 1

  Si frente = 0 O ConvertirANumero(nuevaPrioridad) > ConvertirANumero(MEMORIA_NODOS[frente, 3])
  Entonces
    MEMORIA_NODOS[nuevoNode, 5] <- ConvertirATexto(frente)
    frente <- nuevoNode
  Sino
    actual <- frente
    anterior <- 0
    Mientras actual <> 0 Y ConvertirANumero(MEMORIA_NODOS[actual, 3]) >=
      ConvertirANumero(nuevaPrioridad) Hacer
        anterior <- actual
        actual <- ConvertirANumero(MEMORIA_NODOS[actual, 5])
      FinMientras
    MEMORIA_NODOS[nuevoNode, 5] <- ConvertirATexto(actual)
    MEMORIA_NODOS[anterior, 5] <- ConvertirATexto(nuevoNode)
  FinSi

  Escribir "Proceso encolado dinámicamente."
  Escribir ""
  Escribir "=== RECORRIDO DE VERIFICACIÓN (COLA) ==="
  actual <- frente
  Mientras actual <> 0 Hacer
    Escribir "Dirección [", actual, "] -> ID: ", MEMORIA_NODOS[actual, 1], " | Nombre: ",
    MEMORIA_NODOS[actual, 2], " | Prioridad: ", MEMORIA_NODOS[actual, 3], " | Siguiente apunta a: ",
    MEMORIA_NODOS[actual, 5]
    actual <- ConvertirANumero(MEMORIA_NODOS[actual, 5])
  FinMientras
FinProceso
```



## Pseudocódigo: Desencolar y Ejecutar Proceso (Frente)

Proceso DesencolarEjecutarCPU

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir nodoAEliminar, actual Como Entero

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "101"

MEMORIA\_NODOS[1, 2] <- "Calculadora"

MEMORIA\_NODOS[1, 5] <- "2"

MEMORIA\_NODOS[2, 1] <- "102"

MEMORIA\_NODOS[2, 2] <- "Paint"

MEMORIA\_NODOS[2, 5] <- "0"

frente <- 1

Si frente = 0 Entonces

Escribir "No hay procesos esperando en la CPU."

Sino

nodoAEliminar <- frente

Escribir "EJECUTANDO -> ID: ", MEMORIA\_NODOS[nodoAEliminar, 1], " | Tarea: ",  
MEMORIA\_NODOS[nodoAEliminar, 2]

frente <- ConvertirANumero(MEMORIA\_NODOS[frente, 5])

MEMORIA\_NODOS[nodoAEliminar, 1] <- ""

MEMORIA\_NODOS[nodoAEliminar, 5] <- "0"

Escribir "Ejecución finalizada. El proceso del frente fue retirado de la Cola."

FinSi

Escribir ""

Escribir "\*\*\* RECORRIDO DE VERIFICACIÓN \*\*\*"

actual <- frente

Si actual = 0 Entonces

Escribir "La Cola quedo totalmente vacia (Frente apunta a 0)"

Sino

Mientras actual <> 0 Hacer

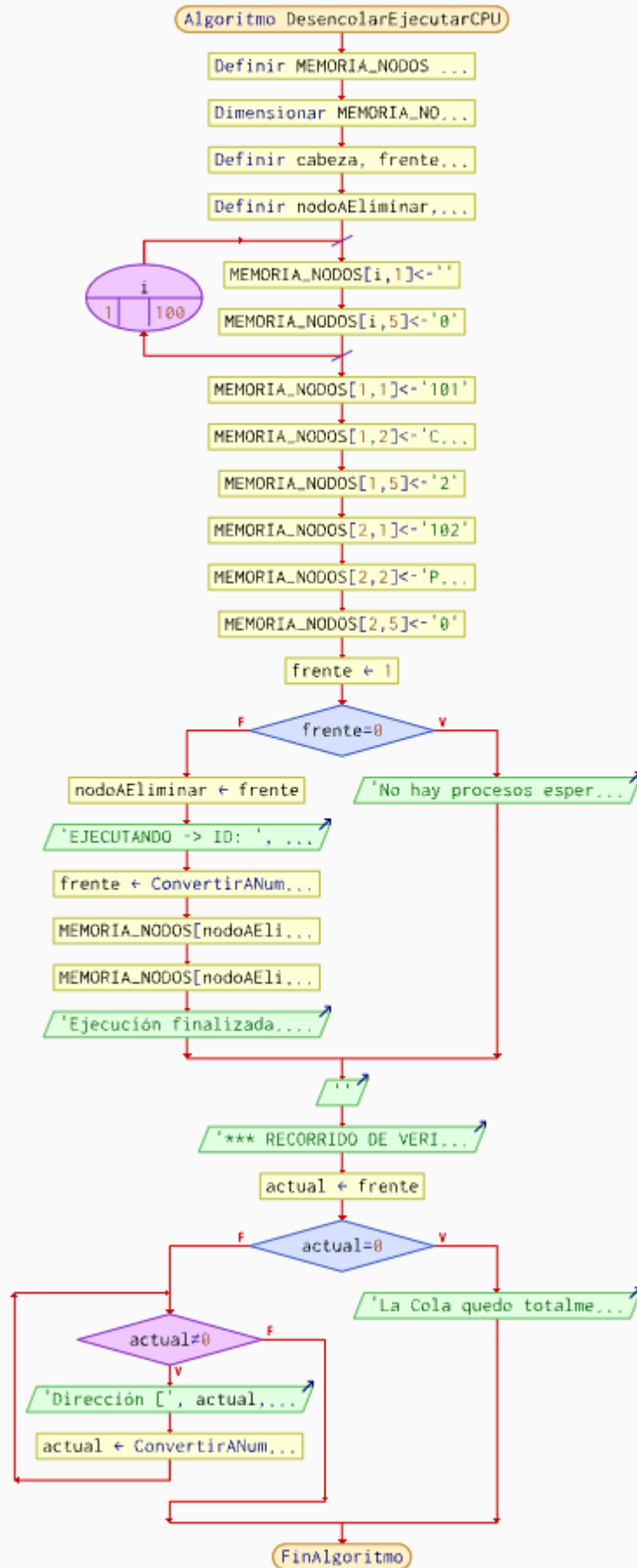
Escribir "Dirección [, actual, "] -> ID: ", MEMORIA\_NODOS[actual, 1], " | Sigue  
esperando: ", MEMORIA\_NODOS[actual, 2], " | Siguiente apunta a: ",  
MEMORIA\_NODOS[actual, 5]

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

FinSi

FinProceso



## Pseudocódigo: Visualizar la Cola de Procesos de la CPU

Proceso VisualizarColaCPU

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir actual, pos Como Entero

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "105"

MEMORIA\_NODOS[1, 2] <- "Word"

MEMORIA\_NODOS[1, 3] <- "5"

MEMORIA\_NODOS[1, 5] <- "2"

MEMORIA\_NODOS[2, 1] <- "106"

MEMORIA\_NODOS[2, 2] <- "Excel"

MEMORIA\_NODOS[2, 3] <- "3"

MEMORIA\_NODOS[2, 5] <- "0"

frente <- 1

Escribir "\*\*\*\* RECORRIDO DE VERIFICACIÓN (VISUALIZAR COLA CPU) \*\*\*\*"

Si frente = 0 Entonces

Escribir "Cola de CPU vacía."

Sino

actual <- frente

pos <- 1

Mientras actual <> 0 Hacer

Escribir "Posición en Cola [", pos, "] | Dirección [", actual, "] -> ID: ",  
MEMORIA\_NODOS[actual, 1], " | Nombre: ", MEMORIA\_NODOS[actual, 2], " |  
Prioridad: ", MEMORIA\_NODOS[actual, 3], " | Enlace Siguiente: ",  
MEMORIA\_NODOS[actual, 5]

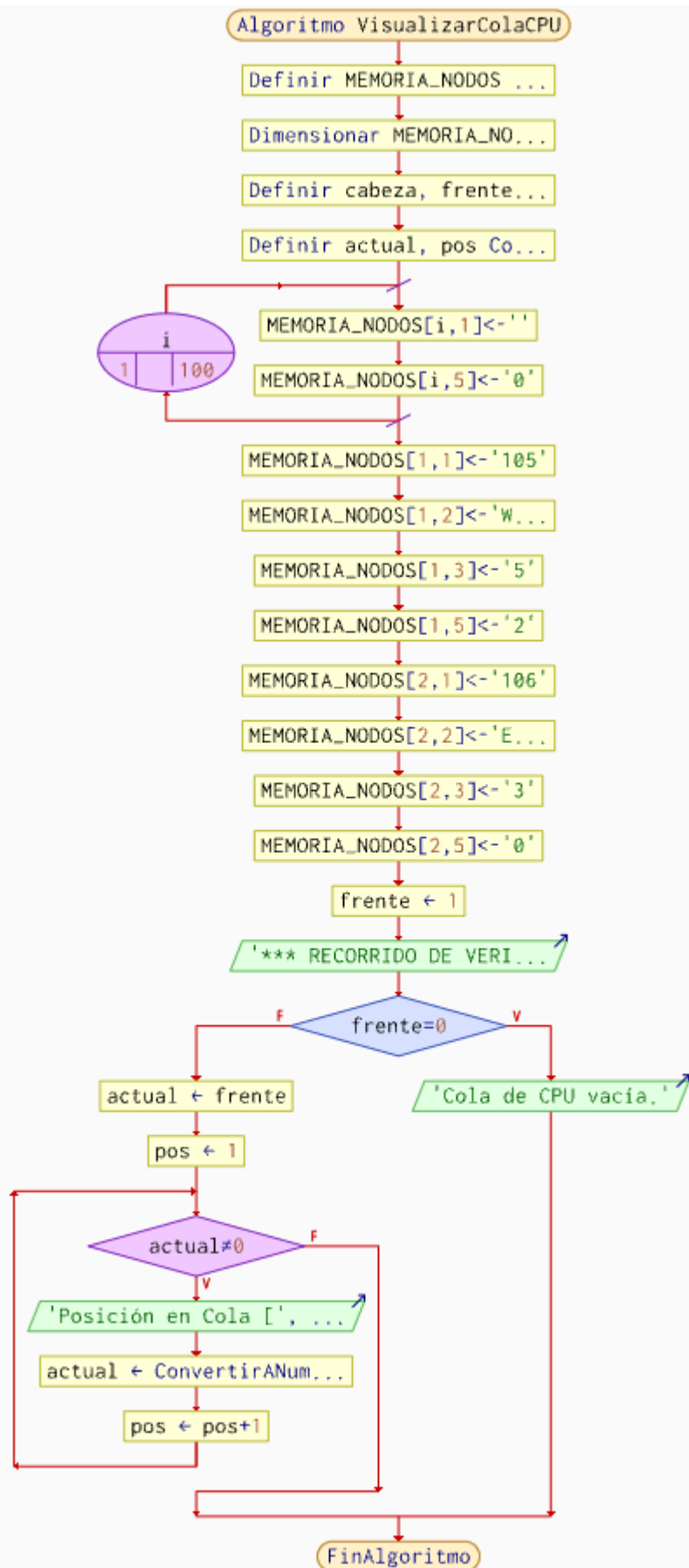
actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

pos <- pos + 1

FinMientras

FinSi

FinProceso



## GESTOR DE MEMORIA (PILA / STACK DINÁMICO )

### Pseudocódigo: Asignación de Memoria (Push)

Proceso AsignarMemoriaPush

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir nuevoBloque, actual Como Entero

Definir idProc, nomProc, tamKB Como Cadena

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "99"

MEMORIA\_NODOS[1, 2] <- "Kernel"

MEMORIA\_NODOS[1, 3] <- "2048"

MEMORIA\_NODOS[1, 5] <- "0"

tope <- 1

ultimoLibre <- 2

Escribir "=== OPERACIÓN APILAR MEMORIA (PUSH) ==="

Escribir "Ingrese ID del proceso:"

Leer idProc

Escribir "Ingrese Nombre del proceso:"

Leer nomProc

Escribir "Ingrese cantidad de KB a asignar:"

Leer tamKB

nuevoBloque <- ultimoLibre

MEMORIA\_NODOS[nuevoBloque, 1] <- idProc

MEMORIA\_NODOS[nuevoBloque, 2] <- nomProc

MEMORIA\_NODOS[nuevoBloque, 3] <- tamKB

MEMORIA\_NODOS[nuevoBloque, 5] <- ConvertirATexto(tope)

tope <- nuevoBloque

ultimoLibre <- ultimoLibre + 1

Escribir "Bloque de memoria apilado."

Escribir ""

Escribir "\*\*\* RECORRIDO DE VERIFICACIÓN (PILA LIFO) \*\*\*"

actual <- tope

Mientras actual <> 0 Hacer

Escribir "Dirección [", actual, "]" (Hacia abajo) -> Proceso: ",

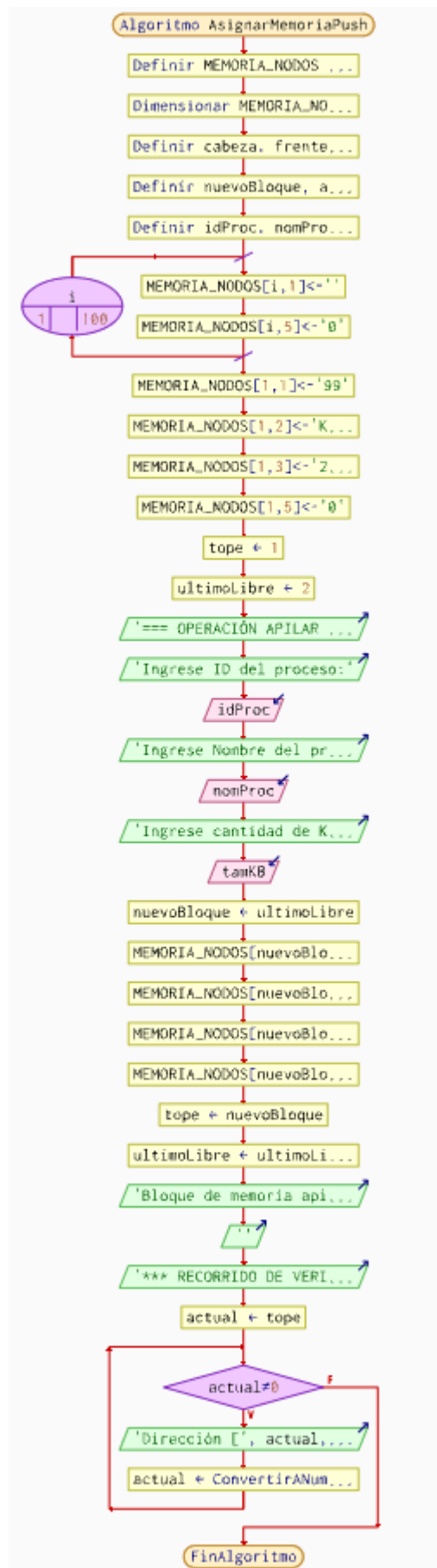
MEMORIA\_NODOS[actual, 2], " | Espacio: ", MEMORIA\_NODOS[actual, 3], " KB | Abajo

apunta a: ", MEMORIA\_NODOS[actual, 5]

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

FinProceso



## Pseudocódigo: Liberación de Memoria (Pop)

Proceso LiberarMemoriaPop

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir bloqueAux, actual Como Entero

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "200"

MEMORIA\_NODOS[1, 2] <- "Sistema"

MEMORIA\_NODOS[1, 3] <- "4096"

MEMORIA\_NODOS[1, 5] <- "0"

MEMORIA\_NODOS[2, 1] <- "300"

MEMORIA\_NODOS[2, 2] <- "VirusTotal"

MEMORIA\_NODOS[2, 3] <- "512"

MEMORIA\_NODOS[2, 5] <- "1"

tope <- 2

Si tope = 0 Entonces

Escribir "Pila de memoria vacía."

Sino

bloqueAux <- tope

Escribir "LIBERANDO TOPE (POP) -> Bloque del proceso: ",

MEMORIA\_NODOS[bloqueAux, 2], " | Tamaño: ", MEMORIA\_NODOS[bloqueAux, 3], " KB"

tope <- ConvertirANumero(MEMORIA\_NODOS[tope, 5])

MEMORIA\_NODOS[bloqueAux, 1] <- ""

MEMORIA\_NODOS[bloqueAux, 5] <- "0"

Escribir "Memoria liberada."

FinSi

Escribir ""

Escribir "\*\*\* RECORRIDO DE VERIFICACIÓN (NUEVO ESTADO PILA) \*\*\*"

actual <- tope

Si actual = 0 Entonces

Escribir "La pila de memoria quedo vacia."

Sino

Mientras actual <> 0 Hacer

Escribir "Tope Actual en Dirección [", actual, "] -> Proceso activo: ",

MEMORIA\_NODOS[actual, 2], " | Consumo: ", MEMORIA\_NODOS[actual, 3], " KB |

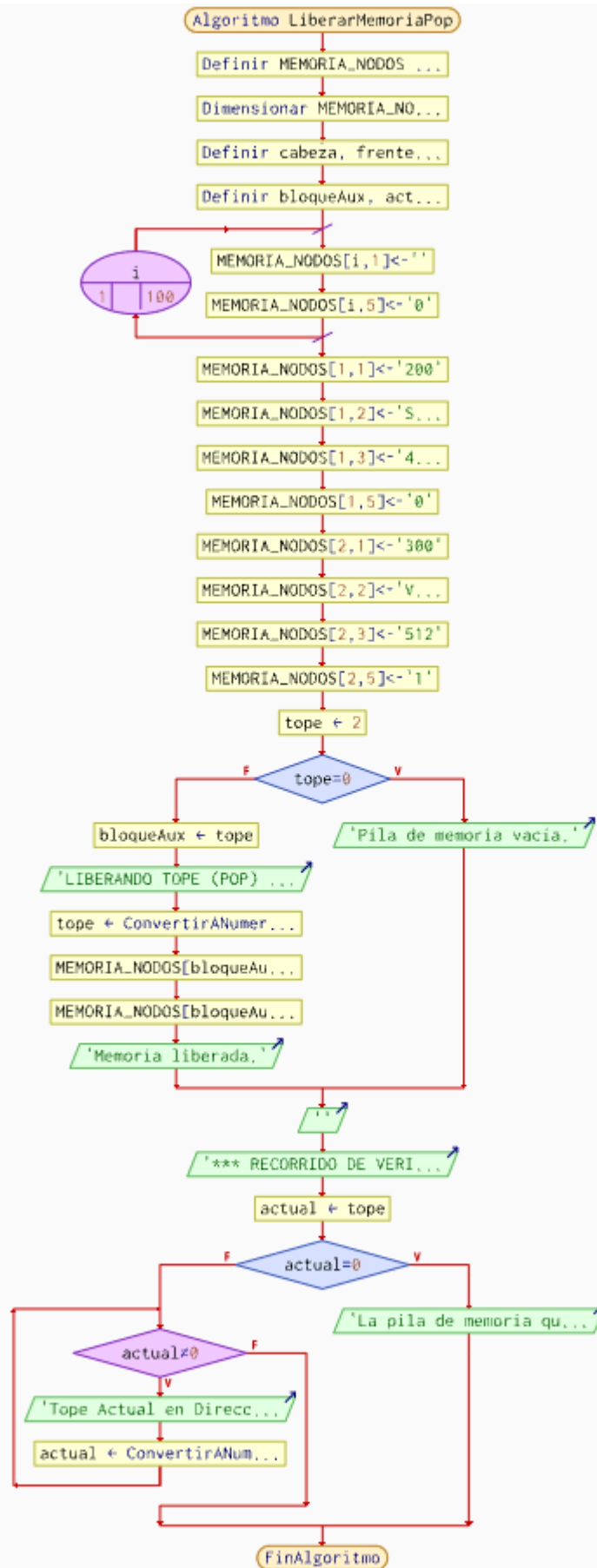
Abajo apunta a: ", MEMORIA\_NODOS[actual, 5]

actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

FinSi

FinProceso



## Pseudocódigo: Visualizar la Pila de Memoria

Proceso VisualizarPilaMemoria

Definir MEMORIA\_NODOS Como Cadena

Dimension MEMORIA\_NODOS[100, 5]

Definir cabeza, frente, tope, ultimoLibre, i Como Entero

Definir actual, totalKB Como Entero

Para i <- 1 Hasta 100 Hacer

MEMORIA\_NODOS[i, 1] <- ""

MEMORIA\_NODOS[i, 5] <- "0"

FinPara

MEMORIA\_NODOS[1, 1] <- "301"

MEMORIA\_NODOS[1, 2] <- "Antivirus"

MEMORIA\_NODOS[1, 3] <- "1024"

MEMORIA\_NODOS[1, 5] <- "2"

MEMORIA\_NODOS[2, 1] <- "302"

MEMORIA\_NODOS[2, 2] <- "Explorador"

MEMORIA\_NODOS[2, 3] <- "512"

MEMORIA\_NODOS[2, 5] <- "0"

tope <- 1

Escribir "\*\*\*\* RECORRIDO DE VERIFICACIÓN (MAPA DE PILA) \*\*\*\*"

Si tope = 0 Entonces

Escribir "No hay memoria reservada (Pila vacía)."

Sino

actual <- tope

totalKB <- 0

Mientras actual <> 0 Hacer

Escribir "Dirección [", actual, "] -> CIMA DE MEMORIA -> Proceso: ",  
MEMORIA\_NODOS[actual, 2], " | Consumo: ", MEMORIA\_NODOS[actual, 3], "  
KB | Enlace Abajo: ", MEMORIA\_NODOS[actual, 5]

totalKB <- totalKB + ConvertirANumero(MEMORIA\_NODOS[actual, 3])

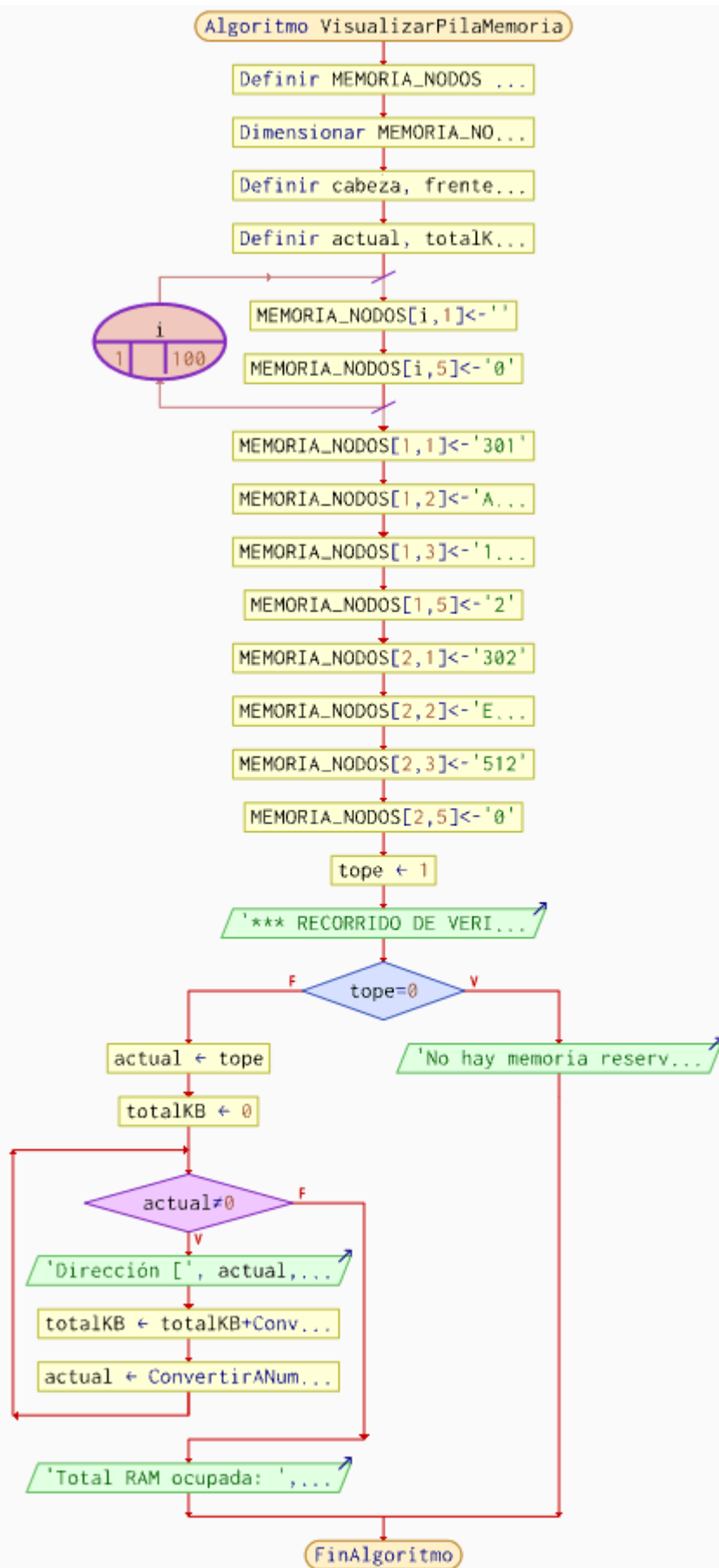
actual <- ConvertirANumero(MEMORIA\_NODOS[actual, 5])

FinMientras

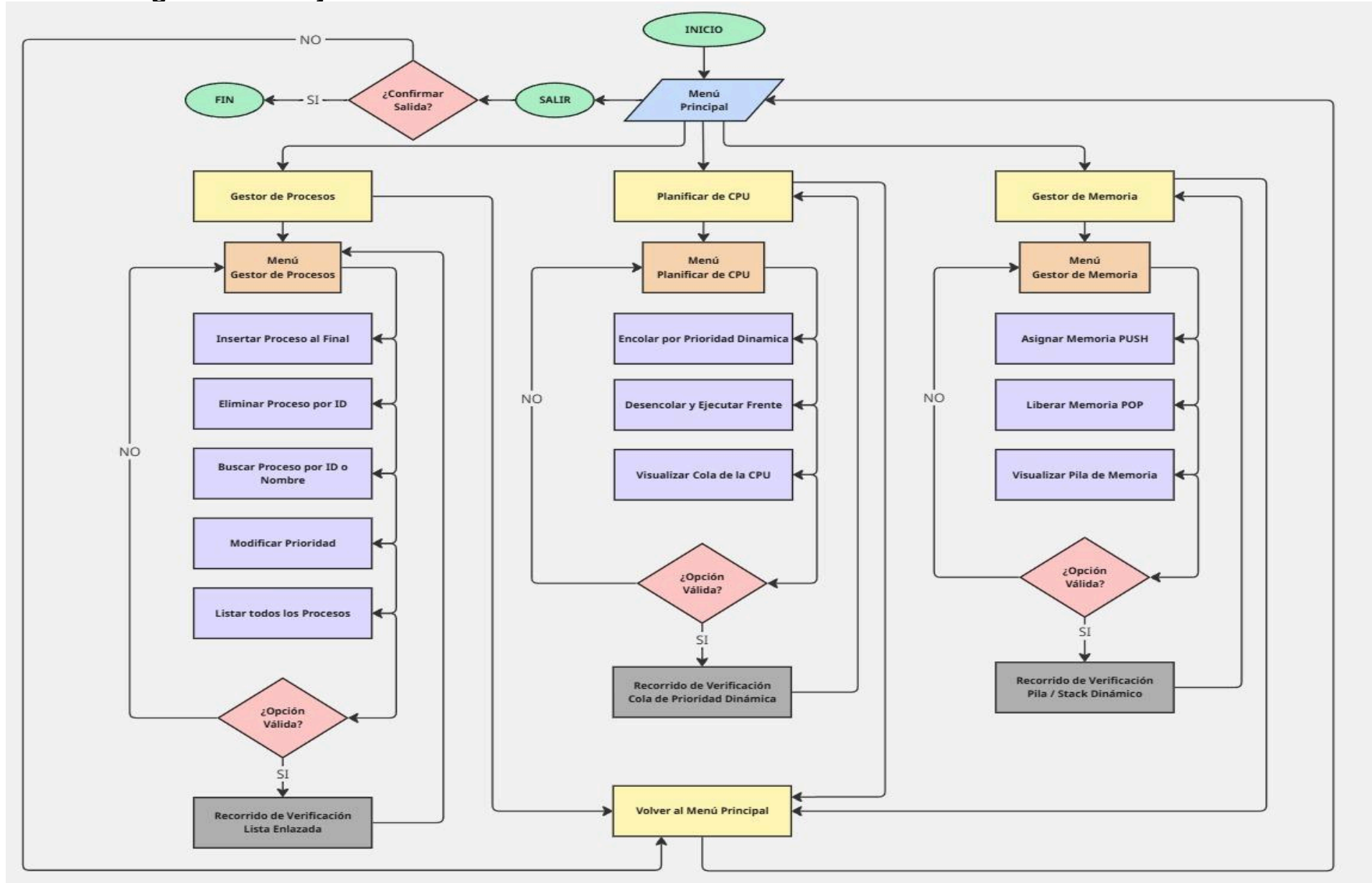
Escribir "Total RAM ocupada: ", totalKB, " KB"

FinSi

FinProceso



## Diagrama de Flujo del Sistema de Gestión de Procesos



## Justificación del diseño

El diseño del sistema se hizo utilizando estructuras de datos dinámicas lineales, porque permiten administrar procesos de una manera más organizada, el uso de estas estructuras ayuda a comprender cómo funciona internamente la administración de procesos y el manejo dinámico de la memoria dentro de un sistema informático. Se utilizaron pseudocódigos y diagramas de flujo para validar la lógica de cada operación antes de su implementación en C++, esto permitió planificar el funcionamiento del programa y facilitar el desarrollo del código.

El sistema fue dividido en tres módulos principales: Gestión de Procesos, Planificador de CPU y Gestión de Memoria, usar esta organización nos permite separar las responsabilidades de cada estructura de datos, facilitando la comprensión del código y el control de las operaciones. Se eligieron estructuras de datos dinámicas porque permiten almacenar información de forma flexible, sin depender de un tamaño fijo de memoria. Gracias a ello, los procesos pueden agregarse, buscarse, modificarse y eliminarse según las necesidades del sistema.

La lista enlazada simple nos sirve para gestionar los procesos registrados, ya que permite recorrer y administrar los nodos eficientemente, la cola de prioridad nos ayuda a organizar la ejecución de los procesos según su nivel de prioridad y la pila fue útil para simular la asignación y liberación de memoria mediante el principio LIFO. Además, el uso de estructuras condicionales y repetitivas permite validar información, controlar el flujo de ejecución y realizar recorridos sobre las estructuras de datos cuando sea necesario, fue así como obtuvimos una solución organizada y acorde con los objetivos planteados para el proyecto.

# **CAPÍTULO 3**

Solución Final

### 3.1 Código limpio, bien comentado y estructurado.

#### Código — Definición de Estructuras de Datos

 CodigoFinal

### 3.2 Capturas de pantalla de las ventanas de ejecución con la prueba técnica.

Menu interactivo:

```
¡Estructuras cargadas correctamente desde 'datos.txt'!  
*****SISTEMA DE GESTION DE PROCESOS*****  
1. Registrar Proceso  
2. Mostrar Lista General de Procesos  
3. Buscar Proceso  
4. Modificar Prioridad de un Proceso  
5. Eliminar Proceso de la Lista  
6. Enviar Procesos 'Listo' a Cola de CPU  
7. Ver Cola de Prioridad  
8. Ejecutar Proceso en CPU  
9. Ver Historial de Memoria  
10. Quitar de la Pila  
11. Guardar Todo  
12. Salir  
Elija una opcion: |
```

Sirve para gestionar procesos en la computadora (crearlos, eliminarlos y ordenarlos) y simula cómo trabaja la CPU y la memoria utilizando estructuras de datos como colas y pilas. Al final, te permite guardar todo en el archivo `datos.txt`

Prueba de la opción 1 (registrar proceso)

```
Elija una opcion: 1  
Ingrese el ID del proceso (Numero entero): 11  
Ingrese Nombre del proceso: Window  
Ingrese Prioridad (Alta, Media, Baja): Alta  
Ingrese cantidad de memoria (KB): 12  
Proceso registrado en la lista con exito.  
Presione una tecla para continuar . . . |
```

Muestra la ejecución de la opción 1 para registrar un nuevo proceso en el sistema. El usuario ingresa manualmente el ID (11), el nombre ("Window"), su prioridad ("Alta") y el consumo de memoria (12 KB), finalizando con un mensaje de éxito que confirma que los datos fueron guardados correctamente en la lista.

### Prueba de la opción 2 (mostrar lista)

```
Eliza una opcion:
2

--- LISTA GENERAL DE PROCESOS ---
ID: 11 | Nombre: Window | Prio: Alta | Memoria: 12 KB | Estado: Listo
Presione una tecla para continuar . . . |
```

La Prueba de la opción 2 (mostrar lista) despliega en pantalla la "Lista General de Procesos" registrados, mostrando detalladamente los atributos del proceso actual (ID: 11, Nombre: Window, Prioridad: Alta, Memoria: 12 KB y Estado: Listo).

### Prueba de la opción 3 (buscar proceso)

```
Eliza una opcion: 3
1. Buscar por ID | 2. Buscar por Nombre: 1
Ingrese ID a buscar: 11
Encontrado -> Nombre: Window | Estado: Listo
Presione una tecla para continuar . . . |
```

La Prueba de la opción 3 (buscar proceso) permite al usuario buscar un proceso específico mediante un submenú; en este caso, se selecciona la búsqueda por ID (opción 1), se ingresa el número 11 y el sistema arroja con éxito que el proceso "Window" fue encontrado en estado "Listo".

### Prueba de la opción 4 (modificar prioridad)

```
Eliza una opcion: 4
Ingrese ID del proceso a modificar: 11
Prioridad actual: Alta
Ingrese nueva prioridad (Alta, Media, Baja): Media
Prioridad cambiada correctamente.
Presione una tecla para continuar . . . |
```

La Prueba de la opción 4 (modificar prioridad) sirve para cambiar la importancia de un proceso existente introduciendo su ID; aquí se

selecciona el ID 11, el programa reconoce que su prioridad actual es "Alta" y permite al usuario cambiarla con éxito a "Media".

#### **Prueba de la opción 5 (eliminar proceso)**

```
Elija una opcion:
5
Ingrese ID del proceso a eliminar: 11
Proceso Window eliminado de la lista.
Presione una tecla para continuar . . . |
```

La Prueba de la opción 5 (eliminar proceso) sirve para borrar un registro del sistema introduciendo su identificador; en el ejemplo se ingresa el ID 11 y el programa confirma que el proceso "Window" fue removido con éxito de la lista general.

#### **Prueba de la opción 6 (enviar proceso a cola)**

```
Elija una opcion: 6
Se enviaron 1 procesos a la Cola de CPU.
Presione una tecla para continuar . . . |
```

La Prueba de la opción 6 (enviar proceso a cola) traslada los procesos en estado de espera hacia la fila de ejecución del procesador; la consola confirma la acción mostrando el mensaje informativo "Se enviaron 1 procesos a la Cola de CPU".

#### **Prueba de la opción 7 (ver cola)**

```
Elija una opcion: 7

--- COLA DE PLANIFICACIÓN DEL CPU ---
[ID: 11 | Proceso: Windows | Prioridad: Baja]
Presione una tecla para continuar . . . |
```

La Prueba de la opción 7 (ver cola) despliega el estado actual de la "Cola de Planificación del CPU", mostrando de forma ordenada los procesos que esperan su turno para ser procesados (en este caso, detalla al proceso "Windows" con ID 11 y Prioridad Baja).

### Prueba de la opción 8 (ejecutar proceso)

```
Elija una opcion:
8
>> [CPU] Ejecutando: Windows...
Proceso terminado exitosamente.
Presione una tecla para continuar . . . |
```

La Prueba de la opción 8 (ejecutar proceso) simula el procesamiento real en el hardware, donde el sistema toma el elemento correspondiente de la cola, muestra la línea "[CPU] Ejecutando: Windows..." y finaliza indicando que el proceso terminó exitosamente.

### Prueba de la opción 9 (ver historial de pila)

```
Elija una opcion: 9

--- HISTORIAL DE MEMORIA LIBERADA (PILA LIFO) ---
<- [ID: 11 | Nombre: Windows | Memoria Liberada: 12 KB]
Presione una tecla para continuar . . . |
```

La Prueba de la opción 9 (ver historial de pila) despliega el "Historial de Memoria Liberada" estructurado como una pila LIFO (Last In, First Out), mostrando el registro del proceso que ha dejado de ejecutarse junto a su ID, nombre y la cantidad de memoria que fue liberada.

### Prueba de la opción 10 (quitar de pila)

```
Elija una opcion: 10
Quitando de la cima de la Pila -> Registro de: Windows
Presione una tecla para continuar . . . |
```

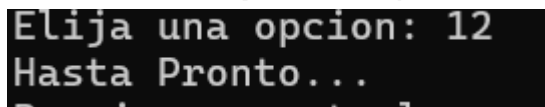
La Prueba de la opción 10 (quitar de pila) ejecuta la eliminación física del elemento que se encuentra en la parte superior o cima de la estructura de datos, notificando en la consola qué registro específico está siendo removido en ese momento.

### Prueba de la opción 11 (guardar datos)

```
Elija una opcion: 11
Datos del sistema respaldados exitosamente en 'datos.txt'.
Presione una tecla para continuar . . . |
```

La Prueba de la opción 11 (guardar datos) realiza una copia de seguridad de toda la información actual del simulador, mostrando un mensaje confirmando que todos los datos del sistema han sido respaldados exitosamente dentro del archivo de texto 'datos.txt'.

## **Prueba de la opción 12 (salir del programa)**



```
Elija una opcion: 12
Hasta Pronto...
```

La Prueba de la opción 12 (salir del programa) detiene por completo la ejecución de la aplicación de consola, mostrando una breve despedida en pantalla ("Hasta Pronto...") antes de cerrar de forma segura la interfaz de comandos.

## **Manual de usuario.**

### **Paso 1: Inicialización del Sistema**

Al ejecutar el programa por primera vez, verán en la consola el menú principal de 11 opciones, si es la primera vez que corre se iniciará limpio, pero si lo usaron antes el programa cargará automáticamente en memoria los procesos guardados en el archivo datos.txt.

### **Paso 2: Registrar Procesos en la Lista**

Presionen la opción 1 y denle Enter.

Ingresen un ID único (ejemplo: 101). Si meten uno repetido el sistema les dará un error.

Ingresen el Nombre del proceso

Ingresen la prioridad y escriban exactamente Alta, Media o Baja.

Ingresen la Memoria en KB (ejemplo: 256000).

El sistema registrará el proceso automáticamente con el estado "Listo" y volverá al menú.

### **Paso 3: Ver, Buscar y Modificar la Lista**

Opción 2: Muestra en pantalla todos los procesos que están en el sistema con sus IDs, memorias y estados actuales (Listo, En Espera o Finalizado).

Opción 3: Les permite buscar un proceso rápido metiendo su ID o escribiendo su nombre exacto.

Opción 4: Si un proceso se vuelve muy importante, elijan esta opción digiten su ID y cámbienle la prioridad (de Baja a Alta, por ejemplo).

Opción 5: Si quieren eliminar por completo un proceso de la lista (simulando que se canceló), digiten su ID y el sistema liberará ese nodo con el comando delete.

#### **Paso 4: Cargar el Planificador de CPU**

Presionen la opción 6, el sistema buscará en la lista general todos los procesos que tengan estado "Listo".

Al pasarlos a la cola, el código evalúa: si el proceso es de prioridad "Alta" lo mandará directo al frente de la cola de espera para que se atienda primero, si es Media o Baja los mandará al final respetando el orden de llegada (criterio FIFO). Los estados en la lista general cambiarán a "En Espera".

Con la Opción 7 pueden ver visualmente cómo quedó ordenada la Cola de CPU.

#### **Paso 5: Simular Ejecución del CPU e Historial**

Presionen la opción 8 (Ejecutar Proceso). El CPU procesará al elemento que esté al frente de la cola (el de mayor prioridad).

Verán un mensaje en consola simulando la ejecución exitosa.

Internamente pasan dos cosas: El estado del proceso en la lista cambia a "Finalizado" y el proceso se apila en el Historial de Memoria.

Presionen la opción 9 para ver el Historial. Verán que los procesos se muestran en estructura de Pila (el último que terminó de ejecutarse aparece arriba del todo en la cima).

Si usan la opción 10 (Desapilar), borrarán únicamente el registro que esté en la cima de la pila (el más reciente).

#### **Paso 6: Guardado Automático y Salida**

seleccionen la opción 11, el programa guardará de inmediato el estado actual de la lista general en el archivo de texto y con la opción 12 se podrá salir del programa

# **CAPÍTULO 4**

Evidencias de trabajo colaborativo

## 1. Repositorio y control de versiones

- Enlace del repositorio:  
<https://github.com/SebastianSota27/TrabajoGrupo06>
- Registro de commits:

### Sota Quispe Sebastian:

| Commits                                                  |                                         |                   |          |
|----------------------------------------------------------|-----------------------------------------|-------------------|----------|
| main                                                     |                                         | SebastianSota27   | All time |
| Commits on Jun 3, 2026                                   |                                         |                   |          |
| Codigo Final acabado                                     | SebastianSota27 authored now            | Verified f4ef5d1f |          |
| InicioCodigoFinal                                        | SebastianSota27 authored 24 minutes ago | Verified 3a83d79  |          |
| Commits on May 28, 2026                                  |                                         |                   |          |
| Se sube el informe del Cap. 2                            | SebastianSota27 authored last week      | Verified a449a2b  |          |
| Descripción de estructuras de datos y operaciones        | SebastianSota27 authored last week      | Verified 2b5a71e  |          |
| Delete TrabajoGrpa/Capitulo2-DescripDatosOperaciones.pdf | SebastianSota27 authored last week      | Verified b2d58d   |          |
| Add files via upload                                     | SebastianSota27 authored last week      | Verified c33b2df  |          |
| Delete TrabajoGrpa/Capitulo2-DescripDatosOperaciones.pdf | SebastianSota27 authored last week      | Verified a3905e7  |          |
| Add files via upload                                     | SebastianSota27 authored last week      | Verified 8838912  |          |
| Commits on May 21, 2026                                  |                                         |                   |          |
| Add files via upload                                     | SebastianSota27 authored 2 weeks ago    | Verified b398892  |          |
| Delete Informe/v0.1                                      | SebastianSota27 authored 2 weeks ago    | Verified 064d8ff  |          |
| Add files via upload                                     | SebastianSota27 authored 2 weeks ago    | Verified 46425e5  |          |
| Delete TrabajoGrpa/Evidencias Parte 1.pdf                | SebastianSota27 authored 2 weeks ago    | Verified 355e8ff  |          |
| Add files via upload                                     | SebastianSota27 authored 2 weeks ago    | Verified 0f239a   |          |
| Commits on May 20, 2026                                  |                                         |                   |          |
| Delete TrabajoGrpa/IER                                   | SebastianSota27 authored 2 weeks ago    | Verified 490b1a   |          |
| Delete Mejorado.cpp                                      | SebastianSota27 authored 2 weeks ago    | Verified 5255e5a  |          |
| Create IER                                               | SebastianSota27 authored 2 weeks ago    | Verified 274776c  |          |
| Delete TrabajoGrupal                                     | SebastianSota27 authored 2 weeks ago    | Verified 205dffe  |          |
| Create TrabajoGrupal                                     | SebastianSota27 authored 2 weeks ago    | Verified 9926df3  |          |
| Create Mejorado.cpp                                      | SebastianSota27 committed 2 weeks ago   | 47d5178           |          |
| Initial commit                                           | SebastianSota27 authored 2 weeks ago    | Verified 2dc6291  |          |

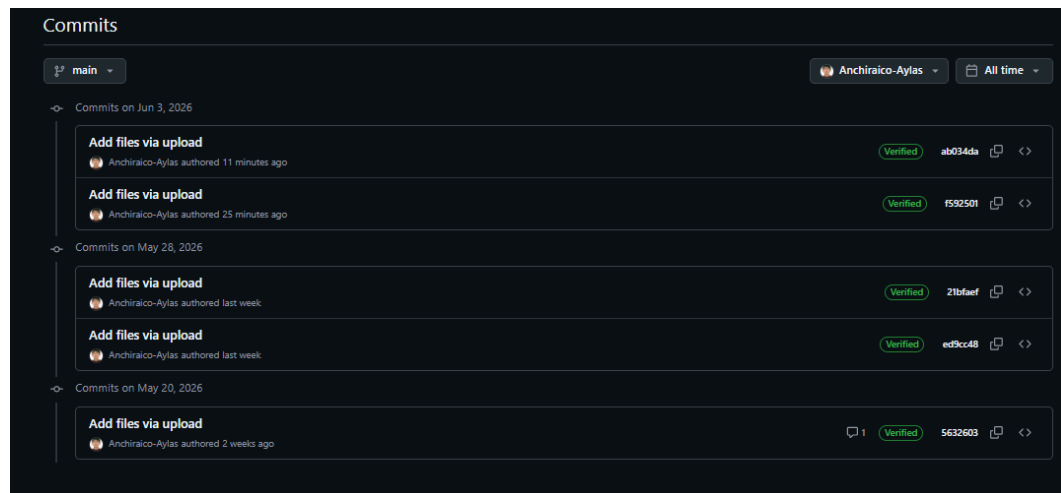
# Cerrón López Hans:

| Commits                                |                                  |                  |          |
|----------------------------------------|----------------------------------|------------------|----------|
| main                                   |                                  | Thancega         | All time |
| Commits on Jun 3, 2026                 |                                  |                  |          |
| UpdateCodigoFinal                      | Thancega authored 22 minutes ago | Verified 3140082 |          |
| Commits on May 28, 2026                |                                  |                  |          |
| Add files via upload                   | Thancega authored last week      | Verified bef43ab |          |
| Add files via upload                   | Thancega authored last week      | Verified 0bc20b3 |          |
| Commits on May 20, 2026                |                                  |                  |          |
| Add files via upload                   | Thancega authored 2 weeks ago    | Verified c215fc3 |          |
| Add Informe version 0.1                | Thancega authored 2 weeks ago    | Verified b4d5c63 |          |
| Create informe                         | Thancega authored 2 weeks ago    | Verified 6c8deff |          |
| Delete Acuerdos Grupales               | Thancega authored 2 weeks ago    | Verified 248e1ed |          |
| Add Acuerdos Grupales section to Actas | Thancega authored 2 weeks ago    | Verified 523559f |          |
| Create Actas                           | Thancega authored 2 weeks ago    | Verified 64950a4 |          |

# Ortega Matos Andy:

| Commits                                   |                                     |                  |          |
|-------------------------------------------|-------------------------------------|------------------|----------|
| main                                      |                                     | JeampierOrtega   | All time |
| Commits on Jun 3, 2026                    |                                     |                  |          |
| Ultima Acta del Trabajo del C2            | JeampierOrtega authored now         | Verified a206002 |          |
| Commits on May 27, 2026                   |                                     |                  |          |
| La ultima acta respecto al segundo avance | JeampierOrtega authored last week   | Verified 0ba0d7c |          |
| Add files via upload                      | JeampierOrtega authored last week   | Verified 42826e0 |          |
| Commits on May 20, 2026                   |                                     |                  |          |
| Actualización del Plan de Trabajo - v1.0  | JeampierOrtega authored 2 weeks ago | Verified 5132766 |          |
| Add files via upload                      | JeampierOrtega authored 2 weeks ago | Verified cb158ee |          |

## Anchiraico Aylas Jesus:



Para este proyecto trabajamos de manera lineal sobre la rama main principal, toda la colaboración se evidencia en el historial directo de commits mencionado anteriormente.

## 2. Plan de Trabajo y Roles Asignados

Enlace del documento colaborativo:

<https://docs.google.com/document/d/1IEPwoiwbxqM7LPOeNKWqw8GPg6JP4ULeg225DrwZX9k/edit?tab=t.0>

## 3. Enlace de la presentación:

<https://canva.link/siiozs60jjjdoe3>